

FluxOps

Continuous and Automated IT Operations

PROJECT REPORT	DevOps / CI/CD / Kubernetes
Application	FluxOps
GitHub	github.com/saad7804/fluxops
Docker Hub	saadmomin7804/fluxops:latest
CI/CD	Jenkins + GitHub Webhook
Deployment	Kubernetes / Minikube
Author	Saad Momin

A practical project demonstrating application development, containerization, CI/CD automation, Docker image publishing, and Kubernetes deployment.

1. Project Overview

FluxOps is an educational DevOps command and deployment assistant designed for students and beginners. It provides a searchable collection of Linux, Git, Docker, Jenkins, AWS CLI and Kubernetes commands, along with practical deployment guides, learning paths and troubleshooting information.

The project was extended into a practical DevOps workflow. The application is containerized with Docker, published to Docker Hub, and deployed to Kubernetes through a Jenkins CI/CD pipeline. A GitHub webhook connects source-code changes to the Jenkins workflow.

Objectives

- Develop a beginner-friendly DevOps learning application.
- Containerize the application using Docker and Nginx.
- Publish the Docker image to Docker Hub.
- Automate build and deployment using Jenkins.
- Deploy the application to Kubernetes using Minikube.
- Verify Kubernetes resources automatically after deployment.
- Demonstrate an end-to-end GitHub → Jenkins → Docker Hub → Kubernetes workflow.

Project Repository

<https://github.com/saad7804/fluxops>

2. Technologies Used

Technology	Purpose
React	Frontend application development
Vite	Development and production build tooling
TypeScript	Type-safe application development
Tailwind CSS	Responsive user-interface styling
Git / GitHub	Version control and source-code hosting
Docker	Application containerization
Nginx	Production web server
Docker Hub	Container image registry
Jenkins	CI/CD automation
Kubernetes	Container orchestration
Minikube	Kubernetes cluster environment
AWS EC2	Cloud host for the DevOps environment

The project combines frontend development with practical DevOps tooling to demonstrate a complete software delivery lifecycle.

3. Application Architecture

The FluxOps application is a frontend application built with React, TypeScript and Vite. The production build is served through Nginx inside a Docker container.

```
Developer
|
v
GitHub Repository
|
v
Jenkins CI/CD
|
+--> Docker Build
|
+--> Docker Hub
|
+--> Kubernetes / Minikube
|
+--> Deployment (2 replicas)
|
+--> NodePort Service
|
v
FluxOps Application
```

The Kubernetes deployment uses the **fluxops** namespace, a Deployment named **fluxops**, and a NodePort Service named **fluxops-service**.

High-level CI/CD workflow

```
GitHub Push
↓
GitHub Webhook
↓
Jenkins
↓
Checkout → Docker Build → Docker Push
↓
Kubernetes Deployment
↓
FluxOps Application
```

4. Docker Configuration

A multi-stage Docker build is used. The Node.js stage installs dependencies and creates the production build. The Nginx stage serves the generated static files.

Docker Image

```
saadmomin7804/fluxops:latest
```

Docker Hub: <https://hub.docker.com/r/saadmomin7804/fluxops>

Dockerfile

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

The same image is used by Kubernetes so that the application deployed through CI/CD matches the container image published by Jenkins.

5. Jenkins CI/CD Configuration

Jenkins automates the complete application delivery process. The pipeline checks out source code, authenticates with Docker Hub, builds the Docker image, pushes the image, deploys Kubernetes manifests and verifies the resulting resources.

Jenkins Credentials

- **docker_creds** – Docker Hub username and Personal Access Token.
- **kubeconfig** – Kubernetes configuration stored as a Jenkins Secret File credential.

Pipeline Stages

Stage	Action
Checkout Code	Clones the main branch from GitHub.
Docker Login	Authenticates Jenkins with Docker Hub.
Build Docker Image	Builds saadmomin7804/fluxops:latest.
Push Docker Image	Publishes the image to Docker Hub.
Deploy to Kubernetes	Creates the namespace and applies Deployment and Service manifests.
Verify Deployment	Checks pods, services, deployment and all resources.

Jenkins therefore acts as the central automation server connecting source control, containerization, image publishing and Kubernetes deployment.

6. GitHub Webhook Integration

A GitHub webhook is implemented to connect repository push events with Jenkins. When changes are pushed to the GitHub repository, the webhook triggers the Jenkins CI/CD workflow.

Webhook endpoint

<http://13.235.95.244:8080/github-webhook/>

Webhook workflow

```
Git Push
↓
GitHub Webhook
↓
Jenkins
↓
Checkout Code
↓
Build Docker Image
↓
Push Docker Image
↓
Deploy to Kubernetes
↓
Verify Deployment
```

The webhook is configured for push events and forms the source-trigger portion of the CI/CD process.

7. Kubernetes Deployment

The application is deployed to Minikube running on an Ubuntu AWS EC2 environment.

Resource	Configuration
Namespace	fluxops
Deployment	fluxops
Replicas	2
Container Port	80
Service	fluxops-service
Service Type	NodePort
NodePort	30007
Container Image	saadmomin7804/fluxops:latest

Verification commands

```
kubectl get pods -n fluxops
kubectl get svc -n fluxops
kubectl get deployment -n fluxops
kubectl get all -n fluxops
```

The final deployment ran two FluxOps pods in the fluxops namespace with both replicas in the Running state.

8. Application Access

The FluxOps Kubernetes Service is exposed using NodePort 30007. During the demonstration, Kubernetes port forwarding was used to make the application accessible through port 9090.

Port-forward command

```
kubect1 port-forward --address 0.0.0.0 service/fluxops-service 9090:80 -n fluxops
```

Application URL

<http://13.235.95.244:9090>

This allowed the deployed FluxOps application to be opened in a browser after the Jenkins pipeline completed successfully.

9. Project Execution Result

The complete CI/CD pipeline executed successfully. The final deployment created the FluxOps Kubernetes namespace, deployed two application replicas, exposed the application through the FluxOps service and verified the Kubernetes resources.

Observed Kubernetes result

```
NAME READY STATUS
fluxops-594858b789-4cx95 1/1 Running
fluxops-594858b789-6dv4 1/1 Running

SERVICE
fluxops-service NodePort 80:30007/TCP
```

Successful result

- Source code was checked out from GitHub.
- Docker authentication completed successfully.
- Docker image was built successfully.
- Docker image was pushed to Docker Hub.
- Kubernetes deployment completed successfully.
- Two application replicas reached the Running state.
- The FluxOps application was accessible in the browser.

10. Project Evidence

The project evidence consists of screenshots documenting the implementation and final results.

Application screenshots

- Home page
- Commands library
- Docker guide
- AWS EC2 guide
- Learning paths
- Saved commands
- About page
- Dark and light modes

DevOps and CI/CD evidence

- Docker installation and image creation
- Minikube and Kubernetes deployment
- Amazon ECR and EKS work
- Jenkins pipeline configuration
- Successful Jenkins build
- Docker Hub image
- Kubernetes pods and services
- FluxOps application after CI/CD deployment
- GitHub webhook configuration

Repository: <https://github.com/saad7804/fluxops>

11. Learning Outcomes

The project provided hands-on experience with the following DevOps concepts and technologies:

- Git and GitHub based source-code management.
- Linux and Ubuntu administration.
- Multi-stage Docker builds.
- Docker Hub image publishing.
- Jenkins credentials and Pipeline syntax.
- GitHub webhook integration.
- CI/CD automation.
- Kubernetes Deployments and Services.
- Kubernetes namespaces and resource verification.
- Minikube cluster management.
- AWS EC2 based DevOps environment.
- End-to-end software delivery automation.

The project demonstrates how individual DevOps tools can be integrated into a single repeatable workflow rather than being used independently.

12. Future Enhancements and Conclusion

Future Enhancements

- Use versioned Docker image tags instead of only the latest tag.
- Add automated unit and integration testing.
- Add code-quality and container security scanning.
- Add Prometheus and Grafana monitoring.
- Use Kubernetes Ingress with HTTPS/TLS.
- Deploy the production workflow to Amazon EKS.
- Add automated rollback for failed deployments.
- Expand the FluxOps learning library and troubleshooting content.

Conclusion

FluxOps demonstrates the practical integration of modern DevOps tools across the software delivery lifecycle. The project starts with a React-based educational application and extends it into a containerized and automated deployment workflow.

GitHub provides source control, the webhook connects source changes to Jenkins, Docker packages the application, Docker Hub stores the image, and Kubernetes runs the application. Jenkins coordinates these stages into a repeatable CI/CD process.

The project therefore provides practical experience with application development, Linux, Docker, Jenkins, GitHub, Docker Hub, Kubernetes, Minikube and AWS EC2.

Author: Saad Momin

GitHub: <https://github.com/saad7804>

LinkedIn: <https://linkedin.com/in/saadmomin>